19 图着色、SIMD 与多线程

> 目标：从正确的单线程顺序冲量出发，逐步回答哪些约束可以同时计算、SIMD 的一条 lane 代表什么，以及线程之间在哪里等待。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 操作次数和加速上界均为理论模型，不是本次测量结果。

19.1 为什么不能直接给接触循环加线程

朴素顺序冲量按照固定顺序处理约束。 处理 A—B 后，B 的速度立刻更新；处理 B—C 时读到的是更新后的 B。 这正是 Gauss-Seidel 式迭代的依赖关系。

教学伪代码，非源码：

```
for constraint in constraints:
    read current body velocities
    compute impulse
    write updated body velocities
```

如果让共享 B 的两条约束同时运行，会发生读写冲突。 例如 B 初始速度为零，两线程分别计算需要增加 1 m/s 和 2 m/s。 若它们都先读到零，再分别写回 1 和 2，最终可能只留下其中一次更新。 这不是可接受的“浮点最后一位不同”，而是丢失了物理作用。

给每个刚体加锁能够避免部分数据竞争，却引入另一个问题。 先拿到锁的是哪条约束，可能随调度变化；约束处理顺序也随之变化。 高连接度刚体还会成为锁争用热点。

因此当前实现首先解决的是依赖组织，而不是先增加线程。

这条热路径为什么值得优化

假设有 100,000 条接触，4 个子步，每个子步分别进行 WarmStart、Solve、Relax。 仅按每条接触一次访问粗略计数，便有：

$$100000 \times 4 \times 3 = 1{,}200{,}000$$

次接触处理。 这个模型尚未计入 Prepare、Store、多接触点、关节等工作。 它说明优化热循环有潜力，却没有说明实际 CPU 时间一定由哪一项主导。

19.2 图着色选择互不冲突的写入集合

把刚体看作图的顶点，把接触和关节看作边。 两条边如果共享会被约束写回的动态刚体，就不能同色。

同色边因此可以并行求解。 不同颜色仍按固定次序处理，让后一个颜色读到前一个颜色完成后的速度。

注意，这里是**边着色**，不是给每个刚体涂一种颜色。 一个刚体连接的多条边可以处于不同颜色。

当前接触与关节使用一致的基本规则：

边类型	搜索颜色	哪些端点占位
动态—动态	0 到 19	两端动态体
动态—静态	22 到 1	只有动态端
动态—运动学	22 到 1	只有动态端
正常颜色找不到	23	进入 overflow

总颜色数为 24，其中最后一色是 overflow。 动态—动态正常只用前 20 色。 单动态端约束从后往前搜索，而且不搜索颜色 0。

它是一种增量贪心着色，不保证最少颜色。 20、21、22 三色不会被动态—动态约束占用，但单动态端约束也可以退到较低颜色。

19.3 逐条手算五条边

设 A、B、C、D 为动态刚体，K 为运动学刚体。 按下表顺序加入五条边，开始时所有颜色均为空。

次序	边	搜索过程	结果
1	A—B	色 0 没有 A、B	0
2	B—C	色 0 已有 B；色 1 可用	1
3	C—D	色 0 只有 A、B，不冲突	0
4	A—K	从色 22 开始，只检查 A	22
5	D—K	色 22 已有 A，但没有 D	22

最终为：

颜色	边集合	动态端占位集合
0	A—B、C—D	A、B、C、D
1	B—C	B、C
22	A—K、D—K	A、D

色 0 的两条边不共享动态端，可以同时求解。 色 22 的两条边共享 K，也仍然允许同色，因为 K 不是约束写回对象。

这正是一个重要版本区别。 旧教程以及 `constraint_graph.c` 文件头旧注释中存在“运动学也占位”的说法。 当前 `b2AddContactToGraph` 和 `b2AssignJointColor` 实际只给动态端设置 bit，应以执行代码为准。

如果把 K 换成静态地面 S，A—S 和 D—S 同样可以同色。 但 static 与 kinematic 的状态来源不同，不能仅用“质量无穷大”把实现细节混为一谈。

19.4 静态与运动学为什么不会写冲突

静态刚体没有 awake `b2BodyState`。 求解时使用局部的 identity 状态提供零速度和单位增量旋转。

运动学刚体有状态，必须提供由用户设置的速度。 否则动态刚体无法正确响应移动平台。 但接触约束不应反过来修改运动学刚体的速度。

当前 SIMD scatter 明确检查 `b2_dynamicFlag`：

```
#if 1
		// I don't use any dummy body in the body array because this will lead to multithreaded
cache coherence problems.
		if ( i1 != B2_NULL_INDEX && ( states[i1].flags & b2_dynamicFlag ) != 0 )
		{
			// [t1.x t1.y t3.x t3.y]
			b2StoreW( (float*)( states + i1 ), _mm_shuffle_ps( t1, t3, _MM_SHUFFLE( 1, 0,
1, 0 ) ) );
		}
```

即使计算得到的运动学速度没有改变，也不能让多个线程“把同样的值再写一次”。 并发写同一地址仍会造成数据竞争和缓存一致性成本。 正确做法是根本不写回这个端点。

同样，不应在状态数组里放一个全线程共用的可写 dummy body，让所有静态 lane 都写它。 本来没有物理依赖的约束，会因此争用一个地址。

图着色消除的是语义上的共享写端。 它不能保证不同动态体不落在同一缓存行，因此伪共享仍可能存在。

实验：让多个动态箱子接触同一运动学平台。 记录着色结果并检查 scatter 条件，确认平台状态仅在适当积分阶段更新，而非由接触线程反复写回。

19.5 为什么必须保留 overflow

固定颜色数不是所有图都能正常着色的保证。 正确实现必须在颜色不足时继续求解，而不是丢掉约束。

设中心动态体 X 连接 21 个不同动态体。 每条边都共享 X，所以不能共色。 前 20 条边可以分别占用 0 到 19。 第 21 条边进入颜色 23 的 overflow，即使 20、21、22 仍空着。

原因是动态—动态边不能侵入为单动态端约束保留的颜色。 这不是颜色查找遗漏。

如果 X 连接的是 23 个静态端，正常可用色为 22 到 1，共 22 色。 第 23 条单动态端约束同样进入 overflow。

overflow 使用单线程标量路径保底。 在 WarmStart、Solve、Relax 中，当前先处理 overflow，再按正常颜色顺序处理。

后颜色最后修改速度，单动态端约束靠后有利于减轻堆叠把物体推入静态地面的趋势。 这是一种求解次序设计，不是“绝对优先级”或“保证无穿透”的证明。

代价：高连接度场景可能有较多串行 overflow，增加线程无法消除这部分成本。

实验：比较大量互不相关的约束与中心星图。 约束数相近时，它们的可并行性可能完全不同，应同时报告颜色分布和 overflow 数量。

19.6 SIMD 的 lane 是不同约束，不是空间维度

二维向量只有 x、y，为什么还可以用四条或八条 lane？ 因为 lane 对应不同接触，而不是一个物体的 x、y、z、w。

先看一个不含角项和 bias 的标量法向求解例。

变量	含义	单位
K	有效逆质量	kg⁻¹
$m_e = 1/K$	有效质量	kg
v_n	相对法向速度	m/s
λ_o	已累计法向冲量	N·s

每条约束执行：

$$\Delta\lambda = -m_e v_n,$$
$$\lambda_n = \max(\lambda_o + \Delta\lambda, 0),$$
$$\Delta\lambda_{\text{used}} = \lambda_n - \lambda_o.$$

四条互不冲突的约束可以一起代入：

lane	K	m_e	v_n	λ_o
0	2	0.5	-2	0
1	1	1	-1	0.2
2	0.5	2	0.5	0.3
3	4	0.25	-4	0

得到：

$$\Delta\lambda = [1, 1, -1, 1]\ \text{N·s},$$
$$\lambda_n = [1, 1.2, 0, 1]\ \text{N·s},$$
$$\Delta\lambda_{\text{used}} = [1, 1, -0.3, 1]\ \text{N·s}.$$

lane 2 正在分离，只能撤回已有的 0.3 N·s，不能产生负累计法向冲量把物体拉回。 向量化没有改变这条物理规则，只把四份相同算式并列执行。

先让标量版本和这个表一致，再谈 SIMD，会比直接读向量 intrinsic 容易得多。

19.7 AoS 状态怎样变成宽寄存器

持久的 `b2BodyState` 是每体一份的 AoS 布局。 SSE2 gather 读取四个状态，再做转置。

输入可以理解成：

物体	第一组字段	第二组字段
1	vx_1、vy_1、ω_1、$flags_1$	Δx_1、Δy_1、$\Delta q.c_1$、$\Delta q.s_1$
2	vx_2、vy_2、ω_2、$flags_2$	Δx_2、Δy_2、$\Delta q.c_2$、$\Delta q.s_2$
3	vx_3、vy_3、ω_3、$flags_3$	Δx_3、Δy_3、$\Delta q.c_3$、$\Delta q.s_3$
4	vx_4、vy_4、ω_4、$flags_4$	Δx_4、Δy_4、$\Delta q.c_4$、$\Delta q.s_4$

转置后的寄存器则分别保存：

- `v.X = [vx₁,vx₂,vx₃,vx₄]`。
- `v.Y = [vy₁,vy₂,vy₃,vy₄]`。
- `w = [ω₁,ω₂,ω₃,ω₄]`。
- 其他增量和标志也按相同方式跨物体排列。

宽约束计算完成，再转置并 scatter 回动态体速度。 所以不能说“Box2D 全部持久数据都是 SoA”。 更准确的说法是：持久状态保持紧凑 AoS，计算阶段使用跨约束的宽布局。

当前平台选择为：

配置	宽度
SSE2	4
NEON	4
当前 WASM SIMD 路径	4
AVX2	8
`BOX2D_DISABLE_SIMD` 的 NONE 路径	4 个标量模拟 lane

关闭 SIMD 不会自动把 `B2_SIMD_WIDTH` 改为 1，也不会关闭图着色。 关节工作块仍分发到各关节求解函数，不能泛称所有关节都已经 SIMD 化。

代价：gather、scatter、转置、非连续读取和分支都消耗时间。 SIMD 宽度是吞吐潜力，不是整个求解器的加速倍数。

19.8 尾 lane 必须显式无害化

某颜色只有 5 条接触，四宽向量需要几组？ 令 N_c 为该色接触数，W 为 lane 数，两者无量纲：

$$N_w = \left\lceil \frac{N_c}{W} \right\rceil .$$

当 $N_c = 5, W = 4$，需要 2 个 wide 槽，共 8 个 lane。 有效利用率为：

$$\eta = \frac{5}{2 \times 4} = 62.5\%.$$

当 $N_c = 10$ 时，四宽利用率为 $10/12 \approx 83.3\%$，八宽为 $10/16 = 62.5\%$。 因此更宽向量在小颜色集合中可能浪费更多尾 lane。

不能为了填满向量而随意把不同颜色混在一次 Solve 中。 这些颜色之间可能共享动态端。

当前处理分为两步：

1. `b2Solve` 在有余数时把最后整个 wide 结构清零。
2. Prepare 对有效 lane 填入接触；空 lane 使用零接触输入，索引保持零。

宽约束的索引使用 `bodySimIndex + 1` 编码。

原含义	wide 索引
真正的 dense index 0	1
真正的 dense index 7	8
`B2_NULL_INDEX=-1`	0
空 lane	0

gather 减 1 后，零索引得到 `B2_NULL_INDEX`，使用 identity 状态而不访问真实 `states[0]`。 scatter 也不会向这个空端写回。

这与公开 ID 的加一编码形式类似，但不是同一个数组、同一种身份。 不能把 wide 索引当作用户的 `b2BodyId`。

实验：分别安排每色 1、3、4、5、7、8、9 条接触。 检查尾槽初始化、索引与越界保护，并比较标量模拟和 SIMD 路径的回归结果。

19.9 stage、block、worker、task 是四个层次

名称	含义
stage	一次算法阶段，例如某颜色的 Solve
block	该阶段的一段工作范围
worker	参与处理的逻辑工人编号
task	交给调度系统执行的回调

一个 worker 可以处理多个 block。 一个 task 可以活过整个约束求解过程。 同一 block 描述还能在后续子步被复用。 不要画成“一个接触等于一个线程”。

当前目标是每个 worker 至多安排约四个块，给工作窃取留余量。 设 worker 数为 P，最大块数目标为 $B = 4P$。 给定工作项数 M 和最小块大小 m：

$$b = \begin{cases} m, & M \le mB, \\ \lceil M/B \rceil, & M > mB, \end{cases}$$

$$N_b = \lceil M/b \rceil.$$

这里 b 为块大小，N_b 为块数，都是工作项计数。 body 的最小块大小为 32；contact 的最小块大小为 4 个 **wide 接触槽**；joint 为 4 条关节。

四个 worker、1,024 个 body 时：

$$B = 16, \quad b = \lceil 1024/16 \rceil = 64, \quad N_b = 16.$$

若有 200 个 wide 接触槽，则块大小为 13，块数为 16，最后一块只有 5 项。 如果只有 5 个 wide 槽，按最小块大小 4 只能产生两块，增加到八个 worker 也造不出八份独立工作。

19.10 当前求解阶段表如何复用

这里只讨论并行调度所需的阶段结构，完整世界步进见第 16 章。

当前 Prepare 在大步开始执行一次，Store 在结束时执行一次。 每个子步重复：

1. 积分速度。
2. 按颜色 WarmStart。
3. 按颜色 Solve。
4. 积分位置。
5. 按颜色 Relax。

Prepare 与 Store 不要求按照颜色串行更新共享速度，可以使用跨颜色的平坦工作范围。 WarmStart、Solve、Relax 则必须遵守颜色边界。

设有效正常颜色数为 C。 当前 `ITERATIONS=1`、`RELAX_ITERATIONS=1` 时，阶段描述数为：

$$N_{\text{stage}} = 5 + 3C.$$

若 $C = 3$，只需要 14 个阶段描述。 4 个子步不会因此分配 56 个描述，而是复用阶段表并增加同步代数。

按执行次数计，开头两次 Prepare、最后一次 Store，再加每子步的 $2 + 3C$ 次阶段调用：

$$3 + 4(2 + 3 \times 3) = 47.$$

47 不是实际屏障次数，因为空阶段直接返回、单块阶段也有快速路径。 `b2SolverTask` 的旧注释仍列出 Restitution，但当前构建和执行阶段表没有独立 Restitution 阶段。

19.11 谁领取块，谁宣布这一色结束

`b2ExecuteStage` 让每个 worker 从分散的起始块开始扫描。 块上带有同步代数 `syncIndex`。

领取过程可以这样理解：

1. 尝试 CAS：只有块仍处于上一代，才能改成本代。

2. CAS 成功者独占执行这个块。
3. 其他 worker 发现它已被领取，就继续寻找其他块。
4. worker 把自己完成的块数原子加到阶段完成计数。

主调度路径 `b2ExecuteMainStage` 分三种情况：

块数	行为
0	直接返回
1	主调度者直接执行
大于 1	发布阶段、参与抢块、等待全部完成

发布信息 `atomicSyncBits` 的低 16 位是阶段索引，高 16 位是同步代数。 body、contact、joint、graph 共享块组各自维护递增代数。 不能简单断言整个 `syncBits` 在所有分组切换中始终数值单调。

代数的目的，是让复用的块区分“这一轮工作”和“上一轮已经完成的工作”。 完成计数达到块总数以后，才能让下一颜色读取更新后的速度。

worker 空等时会使用 pause，并定期 yield。 这些等待都是并行开销的一部分，不是免费同步。

19.12 为什么调用 Step 的线程还要参加

逻辑 worker 0 通过 `mainClaimed` 的 CAS 争夺 orchestrator 身份。 排队 worker 0 和调用 `b2World_Step` 的线程都可能进入这个竞争。 只有获胜者推进阶段，失败者返回。

这样，即使外部任务系统没有及时调度排队的 worker 0，调用线程仍能推动求解。 orchestrator 自己也处理块，因此其他工人无法及时运行时，仍有串行完成路径。

结束时发布 `UINT_MAX`，通知其他 worker 退出。 随后等待非空 task handle 完成，才回收有关临时数据。

通用 `b2ParallelFor` 是另一种实现。 它通过共享的 `nextBlock` 计数领取块，不是复用求解器的每块同步代数协议。 内置 scheduler 又在 task 层使用 Pending→Claimed→Complete 状态，等待任务时调用线程也会帮忙执行可用工作。

层次清楚以后，就不会把“任务系统的任务完成”和“同一求解颜色的屏障完成”混成一回事。

19.13 代价、测量与实验

这组优化的收益来自减少冲突并提高吞吐，但增加了：

- 着色和颜色数组维护。
- SIMD gather、scatter 与尾 lane 浪费。
- 工作块分配和原子领取。
- 颜色之间的等待。
- overflow 的串行保底路径。

可用 Amdahl 模型检查不合理预期。 若假设总时间中 20% 必须串行，80% 可以理想分给 8 个工人，则加速上界为：

$$S \le \frac{1}{0.2 + 0.8/8} = 3.33.$$

这是假设模型，不是 Box2D 的实测倍数。 实际还有屏障、负载不均、内存带宽和缓存开销。

建议测四组场景：

1. 无接触或极少接触，观察调度开销。
2. 大量互不相关的小堆，观察可并行规模。
3. 高连接度星图，观察 overflow 和色间串行。
4. 大量共享运动学平台接触，检查只读端的作用。

每组固定创建顺序、时间步和子步数，只改变 worker 数或 SIMD 配置。 同时记录颜色分布、overflow、wide 槽数量、有效 lane 比例、setup 和 constraints 耗时。 先通过确定性与线程回归测试，再解释速度结果。

19.14 自测与参考答案

1. 五边演算的颜色依次是什么？
2. 共享运动学平台的两条边为什么能同色？
3. 动态星图的第 21 条边去哪里？
4. 5 条接触、四宽向量的利用率是多少？
5. wide 索引 0 是否代表真实 `states[0]`？
6. 三个有效颜色对应多少个当前阶段描述？
7. 四个 worker 处理 1,024 个 body，块大小和块数是多少？

参考答案：

1. 0、1、0、22、22。
2. 只要动态端不冲突，平台状态只读且 scatter 不写回它。
3. overflow 23，不能使用为单动态端保留的正常颜色。
4. 62.5%。
5. 不是；0 是空端编码，真实 dense index 0 编码为 1。
6. $5 + 3 \times 3 = 14$。
7. 每块 64 个 body，共 16 块。

19.15 源码导航

主题	相对源码链接与入口行号
颜色分配	`constraint_graph.c`，67、217

主题	相对源码链接与入口行号
SIMD 平台与宽度	`core.h`，50
SSE2 gather/scatter	`contact_solver.c`，1415、1474
宽接触 Prepare/Solve	`contact_solver.c`，1654、2161
分块计算	`solver.c`，`b2ComputeBlockCount`：769
抢块与阶段等待	`solver.c`，934、973
orchestrator 与 worker	`solver.c`，`b2SolverTask`：1008
阶段表和尾槽初始化	`solver.c`，1255，重点 1333–1514
通用并行循环	`parallel_for.c`，33、64
内置任务调度	`scheduler.c`，55、176

返回 README · 下一章：确定性、精度与性能验证